

App de guardado de webs de inspiración

De la concepción inicial a las decisiones tomadas · 30 de septiembre de 2026

1. Resumen

Este documento recoge el recorrido completo del proyecto: la idea de partida (documento de requisitos v0.1), las decisiones de arquitectura que se han ido tomando, con sus motivos y las alternativas descartadas, y el estado en el que queda la primera versión.

Aspecto	Decisión final
Producto	App personal para guardar webs de inspiración: enlace, título propio, imagen, notas, tags y colecciones.
Front	Vue 3 + Vite + TypeScript, como SPA (sin renderizado en servidor).
Backend	Node + Hono + Drizzle ORM, en TypeScript.
Datos y login	Supabase: PostgreSQL + Supabase Auth (email y contraseña, registro público desactivado).
Imágenes	Disco local en desarrollo; Cloudflare R2 en producción.
Vista previa v1	Imagen subida a mano → og:image de la página → tarjeta con favicon y dominio. La captura automática pasa a la v2.
Diseño	Mínimo y provisional (fondo blanco, bordes negros finos, tipografía básica) hasta validar la plataforma.
Estado	Fase 1 construida. API verificada de punta a punta; pendiente de probar con el proyecto real de Supabase.

2. Punto de partida: la idea

La aplicación permite guardar páginas web interesantes por su diseño, sus componentes o su estilo, para consultarlas después como fuente de inspiración. Es una versión sencilla al estilo de Moodmark. Cada web guardada se representa con una imagen, su enlace, un título propio, tags y notas personales.

El objetivo del MVP es cubrir el ciclo básico: guardar, visualizar en mosaico, buscar y agrupar.

Requisitos y su estado tras las decisiones

ID	Requisito	Prioridad original	Estado en el plan
RF-01	Enlace a la página	Imprescindible	v1 · hecho
RF-02	Título personalizado	Imprescindible	v1 · hecho
RF-03	Captura automática de la parte visible	Imprescindible	Movido a v2. En v1 se sustituye por og:image + imagen manual
RF-04	Captura de página completa	Deseable	v2 automática. En v1 se cubre subiendo la captura a mano
RF-05	Notas	Imprescindible	v1 · hecho
RF-06	Tags	Imprescindible	v1 · hecho (con autocompletado)
RF-07	Mosaico de webs	Imprescindible	v1 · hecho
RF-08	Vista de detalle	Imprescindible	v1 · hecho (muestra la imagen completa si existe)
RF-09	Buscador por tags	Imprescindible	v1 · hecho (varios tags a la vez)
RF-10	Buscador por título	Imprescindible	v1 · hecho
RF-11	Crear colecciones	Imprescindible	v1 · hecho (desde el formulario y el detalle)
RF-12	Gestionar pertenencia a colecciones	Imprescindible	v1 · hecho (y mosaico filtrado por colección)
RF-F1	Guardar componentes concretos	Futuro	Fase 4

3. Cómo se llegó a cada decisión

Las decisiones se tomaron en este orden. Varias cambiaron por el camino al aparecer información nueva; se deja constancia de la opción inicial y del motivo del cambio.

3.1 Sistema de front

- **Análisis de los requisitos:** es una herramienta privada detrás de un login (nadie llega desde Google), casi toda la interacción ocurre en el cliente (mosaico, filtros combinados, detalle, colecciones) y probablemente habrá una extensión de navegador.
- **Conclusión técnica:** una SPA con Vite. Se descartan Next.js, Nuxt, Astro y SvelteKit en modo servidor porque resuelven problemas que aquí no existen (SEO, primera carga, contenido público).
- **Recomendación inicial:** React + Vite, por el tamaño de su ecosistema (masonry, recorte de imágenes para RF-F1, extensiones).
- **Decisión final:** **Vue 3 + Vite + TypeScript**, porque es el framework que mejor conoce la usuaria. Para una app de este tamaño, la soltura con la herramienta pesa más que la diferencia de ecosistema, y Vue cubre todo lo que pide el MVP.

3.2 Backend y dudas abiertas del documento

La usuaria pidió que el backend lo liderase un perfil de ingeniería backend. Las cinco dudas abiertas del documento de requisitos se resolvieron así:

Duda abierta	Respuesta	Motivo
¿Local o en la nube?	Nube	Acceso desde varios dispositivos y un único origen de datos para la web y la futura extensión.
¿Web, extensión, escritorio o combinación?	Web ahora; extensión después	La web da una app usable antes. La extensión se añade usando la misma API.
¿Captura desde servidor o extensión?	Pospuesta a v2	Inicialmente: servidor con Playwright y extensión en una segunda fase. Después se decidió aplazar la captura automática (ver 3.4).
¿Subida manual si la captura falla?	Sí	Siempre habrá webs que bloqueen bots o pidan login. En v1 además es la forma principal de tener una buena imagen.
¿Tags libres o sugeridos?	Libres con autocompletado	Flexibilidad sin duplicados: se normalizan (minúsculas, espacios) y se sugieren los ya usados.

Lenguaje: TypeScript también en el backend, para compartir tipos y validaciones con el front sin aprender otro lenguaje. Framework: Hono, cuyo cliente tipado permite que el front Vue conozca exactamente qué devuelve cada ruta.

3.3 Login y base de datos

- **Propuesta inicial:** Better Auth para el login y Neon como Postgres.
- **Cambio:** la usuaria ya conocía Supabase. Se pasa a Supabase Auth y, de paso, a Supabase como base de datos: menos proveedores, y la parte delicada de seguridad (contraseñas, tokens, límites de intentos) resuelta.
- Supabase se usa solo para el login y como Postgres. El front nunca consulta la base de datos directamente: toda la lógica pasa por la API, que valida el token (JWT) de Supabase.
- Método: email y contraseña (los magic links son incómodos en una extensión). Registro público desactivado: la usuaria se crea desde el panel.
- Imágenes fuera de Supabase Storage: el plan gratuito da 1 GB, frente a los 10 GB sin coste de descarga de Cloudflare R2.

Pega conocida del plan gratuito de Supabase

Los proyectos gratuitos se pausan tras unos 7 días sin actividad. Se reactivan desde el panel sin perder datos. Si llega a molestar, las opciones son el plan Pro (25 \$/mes) o migrar la base de datos, que es un Postgres estándar.

3.4 Captura de pantalla: aplazada a la v2

La captura automática necesita un navegador sin interfaz (Playwright + Chromium) en un servidor, una cola de trabajos, control de páginas enormes, banners de cookies y memoria. La usuaria propuso dejarla para una segunda versión, y se aceptó: reduce mucho la complejidad y el riesgo de la v1.

Orden	Fuente de la vista previa en v1
1	Imagen subida por la usuaria: pegada con Ctrl+V en cualquier parte de la página, arrastrada o elegida del disco. Permite subir capturas de página completa hechas con Firefox.
2	og:image de la página: la imagen que muestran las webs al compartir un enlace. Se copia a nuestro almacenamiento para que no se rompa.
3	Tarjeta generada con el favicon y el dominio. El mosaico nunca tiene huecos.

Limitación asumida

La og:image suele ser una imagen de marketing, no una captura del diseño. Por eso la subida manual se ha hecho lo más cómoda posible.

Consecuencias: desaparecen Playwright, la cola de trabajos y los estados de captura, y ya no es obligatorio un contenedor con Chromium. La v2 solo tendrá que añadir un proceso que rellene los mismos campos de imagen.

3.5 Diseño visual

Se deja para después de validar la plataforma. La v1 usa un estilo mínimo y provisional: fondo blanco, bordes negros finos y tipografía del sistema en negro.

3.6 Entorno de desarrollo

El equipo de la usuaria no tiene Docker ni la CLI de Supabase. En lugar de exigir instalarlos, se ajustó el plan: en desarrollo, la base de datos y el login van contra un proyecto de Supabase en la nube (gratis), y las imágenes se guardan en disco local. El mismo código usa R2 en producción cambiando solo la configuración.

4. Arquitectura final

4.1 Tecnologías

Pieza	Elección	Por qué
Front	Vue 3, Vite, TypeScript, Vue Router, TanStack Query	SPA ligera; caché de datos y filtros guardados en la URL.
API	Hono sobre Node	Ligero, moderno y con cliente tipado para el front.
Validación	Zod (paquete compartido)	Los mismos esquemas en front, API y futura extensión.
ORM	Drizzle	Tipado, cercano a SQL, migraciones sencillas.
Base de datos	PostgreSQL (Supabase) + pg_trgm	Relacional para tags y colecciones; búsqueda por título rápida.
Login	Supabase Auth	Conocido por la usuaria; la API valida el JWT.

Pieza	Elección	Por qué
Imágenes	Disco local / Cloudflare R2 + sharp	Conversión a WebP y miniaturas; R2 sin coste de descarga.
Repositorio	Monorepo con pnpm workspaces	Front, API y código compartido juntos y tipados entre sí.

4.2 Flujo de datos

```
Vue — supabase-js (solo login) → Supabase Auth → devuelve un JWT
|
└─ cliente de Hono + "Authorization: Bearer <JWT>" → API /api/*
                                                    │
                                                    ├─ valida el JWT de Supabase
                                                    └─ PostgreSQL (Supabase)
                                                        └─ Imágenes (disco local / R2)
```

Al guardar una web:

1. La API descarga el HTML de la página (con protección SSRF) y lee título, favicon y og:image.
2. Si no hay imagen propia, copia la og:image, la convierte a WebP y crea la miniatura.
3. Guarda la web con sus tags y colecciones en una única transacción.
4. Si la usuaria adjuntó una imagen, el front la sube después y sustituye a la anterior.

4.3 Estructura del repositorio

```
webs-archive/
├─ apps/
│  └─ api/      Hono + Drizzle · rutas en src/routes · migraciones en drizzle/
│     └─ web/   Vue 3 + Vite · vistas en src/views · componentes en src/components
├─ packages/
│  └─ shared/   esquemas Zod y utilidades comunes
├─ ARCHITECTURE.md  resumen técnico de este documento
└─ README.md       puesta en marcha
```

4.4 Modelo de datos

Tabla	Campos	Notas
webs	id, user_id, url, title, notes, preview_key, preview_source, full_key, site_title, favicon_url, created_at, updated_at	preview_key: miniatura WebP del mosaico. full_key: imagen original subida a mano (p. ej. página completa).
tags	id, user_id, name	Nombre único por usuaria, normalizado. Los tags sin webs se borran solos.
web_tags	web_id, tag_id	Relación muchos a muchos.
collections	id, user_id, name, description	Nombre único por usuaria.
collection_webs	collection_id, web_id	Una web puede estar en varias colecciones.

user_id es el identificador de la usuaria en Supabase. Se preparó desde el principio para que un futuro paso a multiusuario no requiera migraciones. Al buscar por varios tags se devuelven las webs que tienen todos (Y), que es lo útil para buscar inspiración concreta.

4.5 API

Método	Ruta	Función
GET	/api/webs?q=&tags=a,b&collection=	Mosaico con búsqueda y filtros
POST	/api/webs	Guardar web (lee metadatos y og:image)
GET · PATCH · DELETE	/api/webs/:id	Detalle, edición (incluye tags y colecciones) y borrado
PUT	/api/webs/:id/preview	Subir imagen propia
GET · POST	/api/collections	Listar con contador y crear
PATCH · DELETE	/api/collections/:id	Renombrar y borrar (las webs se conservan)
GET	/api/tags	Tags en uso con contador (filtro y autocompletado)

5. Seguridad

Riesgo	Medida
SSRF	El servidor abre URLs que escribe la usuaria. Se bloquean las direcciones no públicas (localhost, redes privadas, metadatos del cloud), comprobándolas al conectar y en cada redirección, con tiempo y tamaño máximos.
API REST de Supabase	La clave pública del front podría leer las tablas. Se activa RLS en todas ellas sin políticas: ese acceso queda cerrado y la API propia no se ve afectada.
Acceso ajeno	Registro público desactivado. Cada consulta filtra por usuaria y no se pueden usar colecciones de otra persona.
Archivos subidos	Se valida el contenido real de la imagen (no la extensión). Tamaño máximo de 25 MB.
Imágenes públicas	Se sirven sin login, pero con rutas aleatorias imposibles de adivinar.

6. Despliegue y costes previstos

Pieza	Servicio	Coste estimado
Front	Cloudflare Pages (estático)	Gratis
API	Railway, Fly.io o Render (Node). Sin Chromium en v1 caben opciones más baratas	0–5 \$/mes
Base de datos + login	Supabase	Gratis (Pro: 25 \$/mes si se necesita)
Imágenes	Cloudflare R2	Gratis hasta 10 GB

En la v2

La captura automática sí exigirá un contenedor con Chromium (unos 5 €/mes). No sirven las plataformas serverless.

7. Estado actual de la fase 1

Construido

- Login, mosaico con buscador por título, filtro por varios tags y por colección.
- Guardar web con tags (autocompletado), colecciones creadas sobre la marcha e imagen por pegado, arrastre o archivo.
- Detalle con edición completa, cambio de imagen y borrado. Página de colecciones para renombrar y borrar.
- Migraciones de base de datos (extensión de búsqueda, tablas y RLS). Documentación: ARCHITECTURE.md y README.md.

Verificado

- Recorrido completo de la API contra un PostgreSQL de prueba y webs reales: 34 comprobaciones superadas (tags, filtros, colecciones, imágenes, validaciones, SSRF, borrado y aislamiento entre usuarios).
- Tipos correctos en los tres paquetes, compilación del front para producción y conexión del front con la API a través del proxy de desarrollo.

Pendiente de verificar

- El front en el navegador con un login real de Supabase.
- La validación de tokens con las claves de un proyecto nuevo de Supabase (en las pruebas se usó el sistema de claves antiguo).
- El almacenamiento en R2 (solo se ha usado el disco local).

Decisiones técnicas menores

- TypeScript 6 en el front: la herramienta de tipos de Vue aún no es compatible con TypeScript 7.
- Sin Pinia: los filtros viven en la URL y no hace falta otro estado global.

8. Hoja de ruta

Fase	Contenido	Estado
1 · Base	Login, CRUD de webs, tags y colecciones, búsqueda, vista previa por og:image o imagen manual	Construida
Diseño	Diseño visual de la plataforma, una vez validado su funcionamiento	Siguiente tras validar

Fase	Contenido	Estado
2 · Captura	Captura automática visible y de página completa (Playwright en un worker)	Planificada
3 · Extensión	Extensión de navegador (WXT + Vue) para guardar con un clic, usando la misma API	Planificada
4 · Componentes	RF-F1: guardar partes concretas de una web con su propia imagen, tags y notas	Futuro

Próximos pasos inmediatos

- Crear el proyecto de Supabase, desactivar el registro público y crear la usuaria.
- Rellenar los archivos .env de la API y del front, aplicar las migraciones (pnpm db:migrate) y arrancar (pnpm dev).
- Probar la app en el navegador con el login real.
- Una vez validada, abrir la fase de diseño visual.